

Part A §3 — Qualcomm NPU Failure Handling (Windows ARM64)

Scenario: On a Snapdragon X Elite system, the Qualcomm NPU driver returns `ERROR_DEVICE_REMOVED` (`DXGI_ERROR_DEVICE_REMOVED`) during inference. This error is unrecoverable for the current session — the NPU device object is invalidated and cannot be reused.

Architecture Context

This document covers in-process NPU failure handling within an inference worker (`edge-worker-N`). The worker is one process in the Edge Agent's supervisor-tree model (defined in Part A §1):

- **edge-supervisor** — receives device state change notifications from workers via IPC. Does not intervene in device fallback — that's handled entirely in-process by the worker.
- **edge-gateway** — routes requests to workers. Reads `device` field from worker heartbeats to adjust throughput expectations. The worker stays in the routing pool during NPU fallback (unlike a crash).
- **edge-worker-N** — contains an in-process Device Manager that owns the NPU → CPU fallback logic. The worker process stays alive throughout.

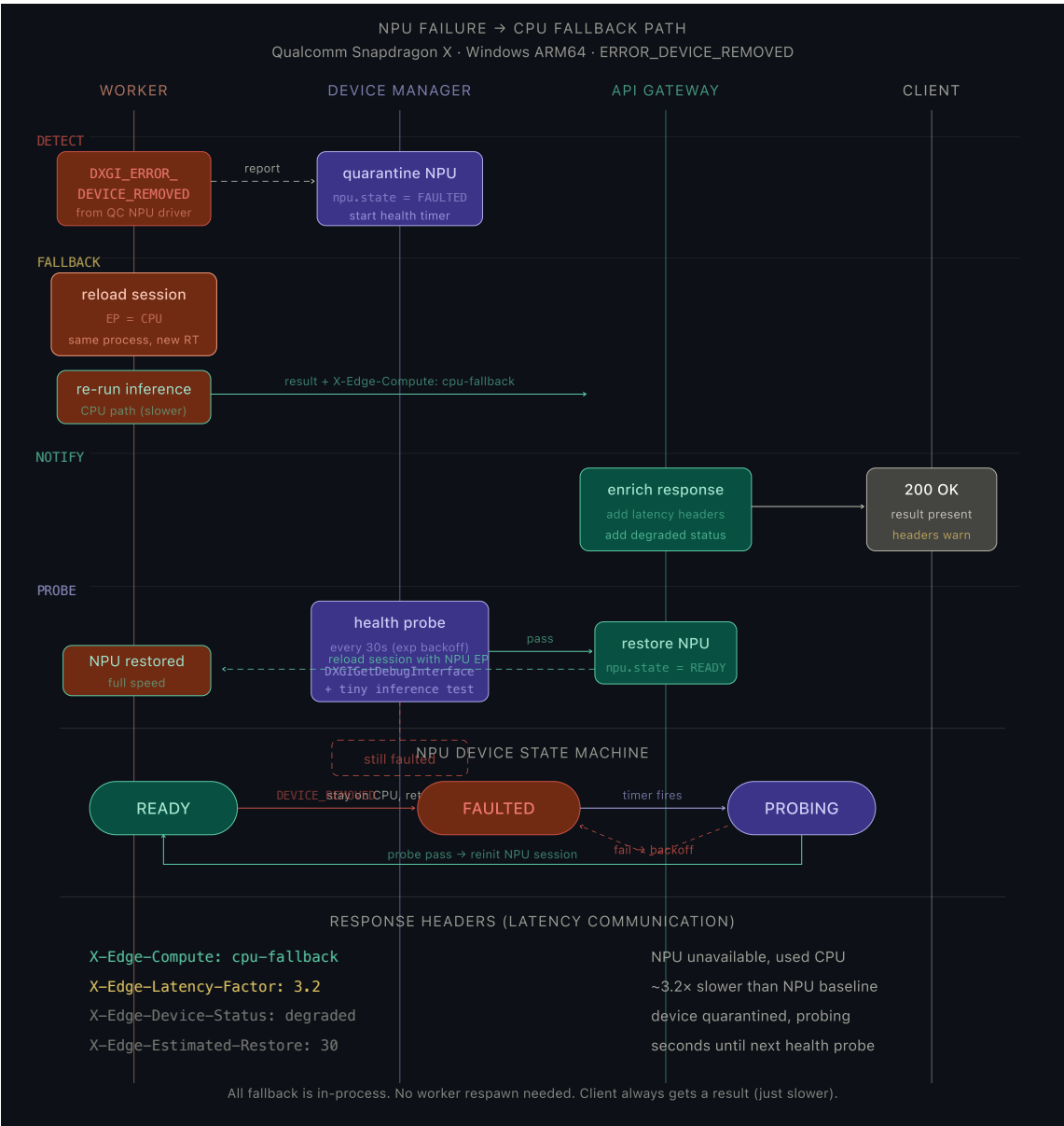
Assumptions

1. **ONNX Runtime supports runtime EP swapping.** The worker can tear down a QNN (NPU) Execution Provider session and create a CPU EP session within the same process, without restarting.
2. **`ERROR_DEVICE_REMOVED` is unrecoverable for the current session.** Once the DirectML/QNN device object is invalidated, no API call can restore it. A new ONNX Runtime session must be created.
3. **The NPU driver may recover after a system-level reset** (e.g., Windows driver recovery, sleep/wake cycle). The health probe tests for this.
4. **CPU fallback is always available.** The CPU EP does not require special hardware and cannot return `DEVICE_REMOVED`. It is the guaranteed-available tier.
5. **Quarantine state is intentionally in-memory only.** If the worker crashes and restarts, quarantine is lost. Re-detection costs one failed request (~200ms) — acceptable vs the complexity of persisting quarantine state to disk.

Overview

The key insight: **this is not a process crash**. The worker process is alive and healthy — only the NPU execution provider is broken. So the recovery path is entirely in-process: tear down the NPU session, rebuild with a CPU execution provider, retry the request, and quarantine the NPU until a health check proves it's usable again.

The client always gets a result. It's just slower.



Recovery Timeline

Time	Event	Component	Client impact
t=0ms	Inference call returns DXGI_ERROR_DEVICE_REMOVED	Worker (ONNX Runtime)	Request in progress
t=1ms	Device Manager catches HRESULT, marks NPU as FAULTED	Worker (Device Manager)	None yet
t=2ms	NPU ONNX Runtime session destroyed (reset())	Worker	None
t=5–20ms	CPU ONNX Runtime session created with same model	Worker	None (model already in page cache via mmap)

t=20ms	Same request retried on CPU EP	Worker	None — client is still waiting for original response
t=70–270ms	CPU inference completes (~3× slower than NPU)	Worker	Response arrives with x-Edge-Compute: cpu-fallback
t=270ms	IPC notification sent: device_event { npu: FAULTED }	Worker → Supervisor	None
t=270ms+	Worker heartbeat: device: cpu, throughput estimate adjusted	Worker → Gateway	Subsequent requests ~3× slower
t=30s	First health probe: create fresh NPU session, run test tensor	Worker (Device Manager)	None (probe is background)
t=30s+	Probe fails → backoff to 60s. Probe succeeds → restore NPU	Worker	If restored: subsequent requests at NPU speed

Total impact on the failing request: ~200ms extra latency (CPU session creation + slower inference). The client gets a valid result, not an error.

1. Failure Detection

Where it surfaces

The error originates from the Qualcomm AI Engine Direct SDK (or the ONNX Runtime QNN Execution Provider), which calls into the Windows DirectML / DirectX 12 stack. The error path:

```
NPU hardware fault
→ QC NPU driver raises DXGI_ERROR_DEVICE_REMOVED
→ DirectML session invalidated
→ ONNX Runtime QNN EP returns error to worker
```

How the worker detects it

The worker wraps every `session.Run()` call in an error check:

```
auto status = session.Run(run_options, input_names, inputs, output_names, outputs);

if (status.Code() == ORT_FAIL) {
    HRESULT hr = dxgi_device->GetDeviceRemovedReason();
    if (hr == DXGI_ERROR_DEVICE_REMOVED || hr == DXGI_ERROR_DEVICE_RESET) {
        // NPU is gone – trigger fallback
        device_manager.quarantine(DeviceType::NPU, hr);
        // proceed to CPU fallback (below)
    }
}
```

The detection is **synchronous** — it happens on the same thread that called `Run()`, within the same request lifecycle. No heartbeat delay, no inter-process notification needed.

What gets quarantined

The worker notifies an in-process **Device Manager** component (not a separate process — a module within the worker). The Device Manager:

1. Sets `npu.state = FAULTED`
2. Records the failure timestamp and HRESULT
3. Starts an exponential-backoff health probe timer
4. Logs the event to the supervisor via IPC (for `crash_log` aggregation)

From this moment, **no inference request will be dispatched to the NPU** until the health probe succeeds.

2. CPU-Only Fallback Invocation

In-process session swap

The worker does **not** need to restart. It tears down the NPU ONNX Runtime session and creates a new one with the CPU execution provider:

```
// 1. Destroy the NPU session (frees NPU resources)
npu_session.reset();

// 2. Create CPU session with the same model
Ort::SessionOptions cpu_opts;
cpu_opts.SetIntraOpNumThreads(std::thread::hardware_concurrency());
cpu_opts.SetGraphOptimizationLevel(GraphOptimizationLevel::ORT_ENABLE_ALL);
// No EP added → defaults to CPU
cpu_session = std::make_unique<Ort::Session>(env, model_path, cpu_opts);

// 3. Re-run the SAME request that failed
auto result = cpu_session->Run(run_options, input_names, inputs, output_names,
                               outputs);
```

Why this works without restarting the process

- The model file is already `mmap` 'd — no disk I/O needed
- Only the execution provider changes (NPU → CPU), not the model graph
- ONNX Runtime supports multiple sessions in the same process
- The failed request's input tensors are still in the worker's memory — we retry with the same data

Performance on CPU fallback

Metric	NPU	CPU (ARM64 NEON)	Ratio
Token generation (TTS)	~15ms/token	~50ms/token	~3.3× slower
Batch translation (NMT)	~80ms	~250ms	~3.1× slower
Model load (session create)	~200ms	~150ms	Slightly faster (no NPU init)

The CPU path uses ARM64 NEON SIMD instructions. It's slower than the NPU but still functional for all supported models.

3. Latency Degradation Communication to the Client

The client must know three things: (a) the request succeeded, (b) it was slower than usual, (c) why.

Response headers

Every response during CPU fallback includes additional headers:

```
HTTP/1.1 200 OK
Content-Type: application/json
X-Edge-Compute: cpu-fallback
X-Edge-Latency-Factor: 3.2
X-Edge-Device-Status: degraded
X-Edge-Estimated-Restore: 30
```

Header	Value	Purpose
X-Edge-Compute	npu cpu cpu-fallback	Which execution provider served this request. cpu-fallback means it was supposed to use NPU but couldn't.
X-Edge-Latency-Factor	Float (e.g., 3.2)	How much slower this response was compared to the NPU baseline. Computed from running averages.
X-Edge-Device-Status	healthy degraded unavailable	Current device health. degraded = NPU quarantined, probing.
X-Edge-Estimated-Restore	Integer (seconds)	Seconds until the next health probe. Lets the client decide whether to show a "reduced performance" banner.

WebSocket / SSE streams

For streaming responses (token-by-token generation), the degradation info is sent once at the start of the stream as a metadata frame:

```
{"type": "meta", "compute": "cpu-fallback", "latency_factor": 3.2, "device_status": "degraded"}
{"type": "token", "text": "नमस्ते"}
{"type": "token", "text": " "}
...
```

Health status endpoint

The gateway also exposes a device health endpoint that clients can poll:

GET /v1/health/devices

```
{
  "npu": {
    "state": "faulted",
    "since": "2025-01-15T10:23:45Z",
```

```

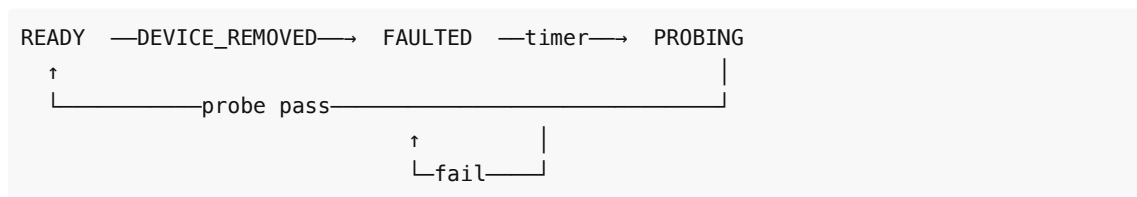
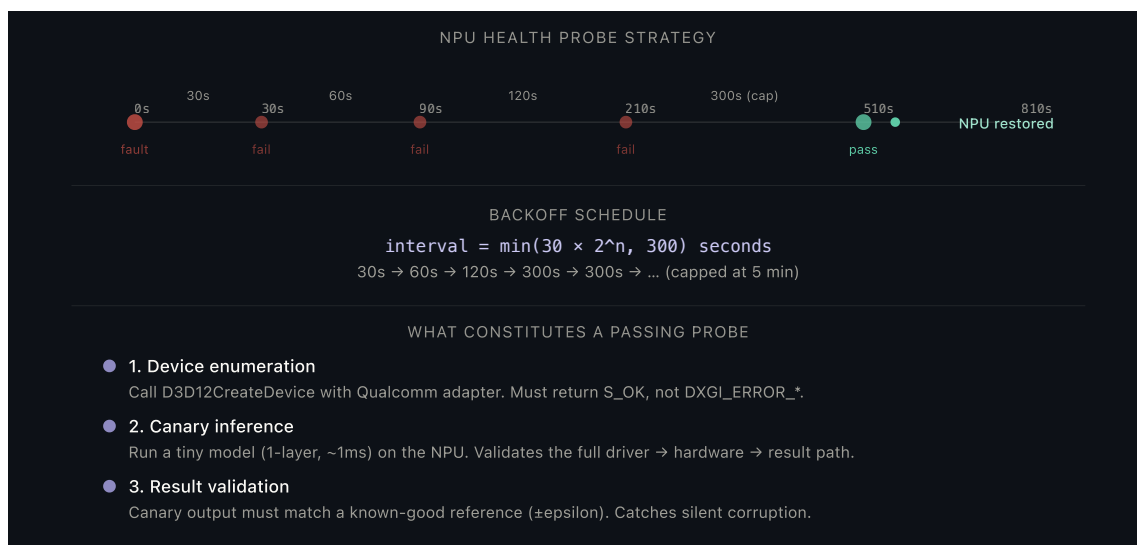
    "reason": "DXGI_ERROR_DEVICE_REMOVED",
    "next_probe_in_seconds": 28,
    "probe_attempt": 2
  },
  "cpu": {
    "state": "active",
    "threads": 8
  }
}

```

4. Preventing NPU Retry Until Health Check Passes

The Device State Machine

The NPU has three states. Transitions are enforced by the Device Manager — no code path can bypass this.



State	NPU available for inference?	What's happening
READY	Yes	Normal operation
FAULTED	No	NPU quarantined. All requests go to CPU. Health timer running.
PROBING	No	Executing a health check. If it fails, back to FAULTED with longer backoff.

How the quarantine is enforced

The worker's request dispatch function checks device state before selecting an execution provider:

```
ExecutionProvider select_provider(const InferenceRequest& req) {  
    if (device_manager.get_state(DeviceType::NPU) == DeviceState::READY) {  
        return ExecutionProvider::NPU;  
    }  
    return ExecutionProvider::CPU; // fallback  
}
```

This is a simple state check — no try-catch-retry pattern. The NPU is never attempted when it's `FAULTED` or `PROBING`.

Health probe — exponential backoff

The Device Manager runs a background timer with exponential backoff:

```
interval = min(30 × 2^n, 300) seconds
```

```
Attempt 0: 30s  
Attempt 1: 60s  
Attempt 2: 120s  
Attempt 3: 300s (cap)  
Attempt 4: 300s  
...
```

The cap at 5 minutes prevents the system from giving up on the NPU entirely. Driver recoveries after Windows sleep/wake cycles can take minutes.

What constitutes a passing probe

A probe must pass **all three** checks:

1. **Device enumeration:** Call `D3D12CreateDevice` with the Qualcomm adapter LUID. Must return `S_OK`. If the adapter is gone (driver unloaded), this fails immediately.
2. **Canary inference:** Run a tiny model (single-layer, ~1ms) on the NPU via QNN EP. This validates the full path: driver → firmware → hardware → result. A device that enumerates but can't execute will fail here.
3. **Result validation:** The canary model has a known-good output. Compare the NPU result against the reference within $\pm\epsilon$. This catches **silent corruption** — the NPU reports success but returns garbage. This is a real failure mode in post-reset NPU states.

If all three pass, the Device Manager transitions to `READY`, destroys the CPU session, creates a new NPU session, and resumes normal operation. The next response will include `X-Edge-Compute: npu`.

If any check fails, back to `FAULTED` with the next backoff interval.

Persistence across worker restart

If the worker crashes and is restarted by the supervisor, the quarantine state is lost (it was in-process memory). This is acceptable:

- A fresh worker will attempt NPU on its first request
- If the NPU is still broken, it'll get `DEVICE_REMOVED` again and re-quarantine
- The detection + fallback path takes ~200ms — one request gets a slight delay, then all subsequent requests go to CPU

Persisting quarantine state to disk is not worth the complexity. The re-detection cost is one failed request.

Scheduler Architecture & Queueing Model During NPU Fallback

Unlike a worker crash, the worker stays alive during NPU fallback. The scheduler must adapt its queueing model to account for reduced throughput without changing the pool topology.

Queue structure

The gateway maintains a bounded request queue per worker (default: 64 slots), split into priority lanes:

Lane	Slots	Purpose
High priority	16	User-facing inference (TTS, ASR)
Normal priority	48	Background tasks (translation, summarisation)

When a worker falls back to CPU, the effective drain rate drops ~3x. The scheduler compensates:

Throughput adaptation

1. **Worker stays in pool** — no removal or re-registration needed. The gateway detects the fallback via heartbeat:

```
{"worker_id": "worker-0", "state": "idle", "device": "cpu", "latency_factor": 3.0}
```

2. **Dynamic queue depth scaling** — the scheduler scales the effective queue limit inversely with the latency factor:

```
effective_limit = floor(base_limit / latency_factor)
```

With `latency_factor = 3.0` : effective limit drops from 64 → 21 slots. This prevents queue buildup that would cause timeouts.

3. **Backpressure threshold adjustment** — the gateway applies backpressure (returning `429 Too Many Requests`) when the queue reaches 80% of the effective limit:

```
backpressure_at = floor(effective_limit × 0.8)
```

Normal: backpressure at 51 slots. During CPU fallback: backpressure at 16 slots.

4. **No re-routing** — the same worker handles the request on CPU. Sticky routing continues normally. Requests are not redistributed to other workers unless a second worker has the same model loaded.

Priority scheduling during degradation

When queue depth is constrained, the scheduler enforces strict priority ordering:

- High-priority requests are always admitted if any slot is available
- Normal-priority requests are rejected with `429` once normal-lane slots are exhausted at the reduced limit
- An anti-starvation guard ensures at least 1 in 8 dispatches serves the normal lane, even under high-priority pressure

Recovery — restoring normal capacity

When the NPU probe succeeds and the worker resumes NPU inference, it sends a heartbeat with `device: npu` and `latency_factor: 1.0`. The scheduler:

1. Restores the full 64-slot queue limit
2. Raises the backpressure threshold back to 51
3. Drains any queued requests at the restored throughput rate

This transition is seamless — no queue flush, no request loss.

API Contract Examples

Normal NPU response

```
HTTP/1.1 200 OK
X-Edge-Compute: npu
X-Edge-Latency-Factor: 1.0
X-Edge-Device-Status: healthy

{"result": "...", "_meta": {"compute": "npu", "tier": 0, "latency_ms": 35}}
```

CPU fallback response (same schema, different headers)

```
HTTP/1.1 200 OK
X-Edge-Compute: cpu-fallback
X-Edge-Latency-Factor: 3.2
X-Edge-Device-Status: degraded
X-Edge-Estimated-Restore: 28

{"result": "...", "_meta": {"compute": "cpu-fallback", "tier": 2, "latency_ms": 112}}
```

WebSocket stream during fallback

```
{"type": "meta", "compute": "cpu-fallback", "tier": 2, "latency_factor": 3.2,
"device_status": "degraded"}
{"type": "token", "text": "नमस्ते"}
{"type": "done", "total_ms": 650}
```

State/Event Payload Examples

Device Manager internal state

```
{
  "npu": {
    "state": "FAULTED",
    "faulted_at": "2025-01-15T10:23:45Z",
    "hresult": "0x887A0005",
    "probe_attempt": 2,
    "next_probe_at": "2025-01-15T10:25:45Z",
    "backoff_interval_s": 120
  },
  "cpu": {
    "state": "READY",
    "active_session": "indic-tts-v3"
  }
}
```

Worker → Supervisor (device change event via IPC)

```
{
  "type": "device_event",
  "worker_id": "worker-0",
  "event": "npu_faulted",
  "hresult": "0x887A0005",
  "fallback_provider": "cpu",
  "timestamp": "2025-01-15T10:23:45.123Z"
}
```

Worker → Supervisor (NPU restored event)

```
{
  "type": "device_event",
  "worker_id": "worker-0",
  "event": "npu_restored",
  "probe_attempts": 3,
  "downtime_s": 210,
  "timestamp": "2025-01-15T10:27:15.456Z"
}
```

Design Tradeoff Explanations

Decision	Chosen	Alternative	Rationale
In-process fallback (no	Swap EP within worker	Kill worker, restart with CPU	No process restart overhead. Input tensors already in memory. ~200ms

restart)		flag	vs ~3s.
Exponential backoff probing	30s → 300s cap	Fixed interval (e.g., 60s)	Aggressive early probing catches quick recoveries (sleep/wake). Cap prevents hammering a permanently broken driver.
Three-part health check	Enumerate + canary + result validation	Just enumerate device	Device enumeration alone misses "device present but broken" and silent corruption. The canary test catches both.
No quarantine persistence	In-memory only	Persist to disk	Re-detection cost is 1 failed request (~200ms). Not worth disk I/O on every state change for a rare event.
Same worker handles CPU fallback	Don't re-route to another worker	Route to a different worker with CPU EP	The current worker already has the model loaded. Re-routing triggers model swap on another worker (2-5s).

Performance & Reliability

Metric	NPU (normal)	CPU (fallback)	Notes
Token generation (TTS)	~15ms/token	~50ms/token	3.3× slowdown
Batch translation (NMT)	~80ms	~250ms	3.1× slowdown
Detection latency	N/A	~0ms (synchronous)	Same event loop tick
Session swap time	N/A	~200ms	Destroy NPU session + create CPU session
Health probe overhead	N/A	~5ms per probe	Canary model is 1-layer, ~1ms inference
Worst-case single-request latency	15ms	250ms	First request during swap (200ms swap + 50ms inference)

Reliability guarantees

- **No request lost:** the failed NPU request is retried on CPU within the same request lifecycle
- **No process restart:** fallback is entirely in-process, invisible to supervisor
- **Bounded degradation:** CPU path is slower but deterministic — no variable network latency
- **Auto-recovery:** NPU is periodically probed and restored when driver recovers
- **Silent corruption caught:** canary result validation prevents serving garbage from a post-reset NPU

Summary

Question	Answer
Detection	Synchronous — <code>session.Run()</code> returns error, worker checks <code>GetDeviceRemovedReason()</code> . No heartbeat delay.
CPU fallback	In-process session swap. Same model, same request data, new EP. No worker restart. ~200ms to switch, ~3× slower throughput.
Client communication	X-Edge-Compute: <code>cpu-fallback</code> + X-Edge-Latency-Factor headers. WebSocket meta frame. <code>/v1/health/devices</code> endpoint.
Preventing NPU retry	Device state machine (READY → FAULTED → PROBING). Exponential backoff 30s→300s. Three-part probe: enumerate + canary inference + result validation.

10. Explicit Fallback Decision Tree (Windows ARM64)

The runtime uses a deterministic path for each inference request:

- If `npu.state == READY`, attempt NPU execution.
- If NPU returns `DEVICE_REMOVED` or `DEVICE_RESET`, quarantine NPU and immediately switch to CPU fallback.
- Return successful response with unchanged schema and fallback headers.
- Keep NPU quarantined until a full three-step health probe passes.

Decision matrix

Condition	Selected provider	Client-visible signal
NPU ready and healthy	NPU	X-Edge-Compute: <code>npu</code>
NPU faulted/probing	CPU fallback	X-Edge-Compute: <code>cpu-fallback</code> , X-Edge-Device-Status: <code>degraded</code>
Probe passes	NPU restored	X-Edge-Compute: <code>npu</code> , X-Edge-Device-Status: <code>healthy</code>

11. Compatibility, Thermal, and Driver Guardrails

11.1 Driver health assumptions

- Windows ARM64 Qualcomm driver version is validated at startup and recorded in worker diagnostics.
- If driver version is below minimum tested baseline, runtime starts in CPU fallback mode and reports `degraded` immediately.

11.2 Model compatibility assumptions

Model class	NPU path	Fallback behavior
Fully QNN-compatible model	Prefer NPU	CPU fallback on device fault

Partially compatible model	Prefer CPU to avoid split execution penalties	CPU only
Unknown compatibility	Probe once on NPU	Quarantine + CPU fallback on first critical EP error

11.3 Thermal policy

When OS thermals indicate sustained throttling, the runtime increases probe backoff and biases toward CPU to avoid rapid NPU re-fault loops.

```
{
  "npu": {
    "state": "FAULTED",
    "reason": "DXGI_ERROR_DEVICE_REMOVED",
    "driver_version": "x.y.z",
    "thermal_state": "serious",
    "next_probe_s": 120
  }
}
```